

Unit-4: Angular JS

4.1 Concepts and characteristics of Angular JS

- AngularJS is an **open-source web application framework**. It was originally developed in **2009** by **Misko Hevery** and **Adam Abrons**. It is now maintained by Google. Its latest **version is 1.2.21**.
- AngularJS is a **JavaScript framework** that helps build web application.
- It can be added to an HTML page with a `<script>` tag.
- AngularJS extends HTML attributes with **Directives**, and binds data to HTML with **Expressions**.
- AngularJS changes static HTML to dynamic HTML. It extends the ability of HTML by adding built-in attributes and components and also provides an ability to create custom attributes using simple JavaScript.

Why to Learn AngularJS?

- AngularJS is a efficient framework that can create Rich Internet Applications (RIA).
- AngularJS provides developers an options to write client side applications using JavaScript in a clean Model View Controller (MVC) way.
- Applications written in AngularJS are cross-browser compliant. AngularJS automatically handles JavaScript code suitable for each browser.
- AngularJS is open source, completely free, and used by thousands of developers around the world. It is licensed under the Apache license version 2.0.

Examples of AngularJS Website

1. Upwork
2. Colgate
3. Adobe Fonts
4. Firebase
5. PayPal Community
6. Google Play Books
7. Fitbit Community

Features of AngularJS

1. Data-binding

- It is the automatic synchronization of data between model and view components.

2. Scope

- These are objects that refer to the model. They act as a glue between controller and view.

3. Controller

- These are JavaScript functions bound to a particular scope.

4. Architecture

- An angular application is built using MVC architecture that stands for Model View and Controller.
- It separates the application into three parts model part, view part and controller part as per the components of MVC architecture.
- Using this, architecture presentation part, logic part and application data part is split into the separate section which allows managing of application in a very fluent manner.

5. Services

- AngularJS comes with several built-in services such as \$http to make a XMLHttpRequests. These are singleton objects which are instantiated only once in app.

6. Filters

- These select a subset of items from an array and returns a new array.

7. Directives

- Directives are markers on DOM elements such as elements, attributes, css, and more.
- These can be used to create custom HTML tags that serve as new, custom widgets.
- AngularJS has built-in directives such as ngBind, ngModel, etc.

8. Templates

- These are the rendered view with information from the controller and model.
- These can be a single file (such as index.html) or multiple views in one page using *partials*.
- The template tells us to which part we want to load on which position.

9. Routing

- It is concept of switching views.

10. Dependency Injection

- This built-in injection helps in developing the application easily as well as it is easy to understand.
- It helps an application easier to test.
- Whenever angular JS detect that you need a service then it immediately provides an instance for that.
- It allows you to ask for your dependencies rather than having to go look for them or making it by yourself.

11. Code Less

- A programmer can write less and perform more functionality with the same code.
- Filters in angular provide the functionality of write less do more. The various filters in angular are uppercase, lowercase, currency etc. You can use the filter and easily format the data.

12. Not Browser Specific

- Angular applications are not browser specific means there is no browser constraint on an angular application.
- It can run on all major browsers except internet explorer 8.0 and smartphones including Android and ios based phones/tablets.
- It claims to support “CLASS A” browsers such as Chrome, Firefox, Safari, IE8, Android.

Advantages of AngularJS:

- **Dependency Injection:** Dependency Injection specifies a design pattern in which components are given their dependencies instead of hard coding them within the component.
- **Two way data binding:** AngularJS creates a two way data-binding between the select element and the orderProp model. orderProp is then used as the input for the orderBy filter.
- **Testing:** Angular JS is designed in a way that we can test right from the start. So, it is very easy to test any of its components through unit testing and end-to-end testing.
- **Model View Controller:** In Angular JS, it is very easy to develop application in a clean MVC way. You just have to split your application code into MVC components i.e. Model, View and the Controller.

Disadvantages of AngularJS:

- **Not Secure** – Being JavaScript only framework, application written in AngularJS are not safe. Server side authentication and authorization is must to keep an application secure.
- **Not degradable** – If the user of your application disables JavaScript, then nothing would be visible, except the basic page.

Use cases of AngularJS

We have already established that AngularJS is a perfect choice for creating dynamic front-end applications. Let's look at some more use cases of the AngularJS framework.

It will help you decide if it is a perfect fit for your next web development project.

Dynamic applications

AngularJS has frameworks to build simple but powerful dynamic web applications with scalability and security.

Enterprise web applications

There's a lot that goes into building enterprise web applications, such as architecture, localization, theming, build, packaging, best practices, and more. However, features such as refactoring, navigation, auto completion, etc., enable AngularJS developers to build large-scale app development projects smoothly.

Custom development

Highly specialized businesses require specialized features and components custom-tailored to the company's goals and objectives. AngularJS is an ideal choice for building on-demand applications due to its wide variety of features.

Video-streaming applications

With its stable, extensible, and advanced features, AngularJS developers can build video-streaming apps that offer ultra-modern and seamless experiences to the end-users.

E-commerce applications

AngularJS offers fluid UI standards combined with robust UI controls enabling AngularJS developers to bind complex two-way data with the best architectural design to help your company significantly impact the market.

Content management applications

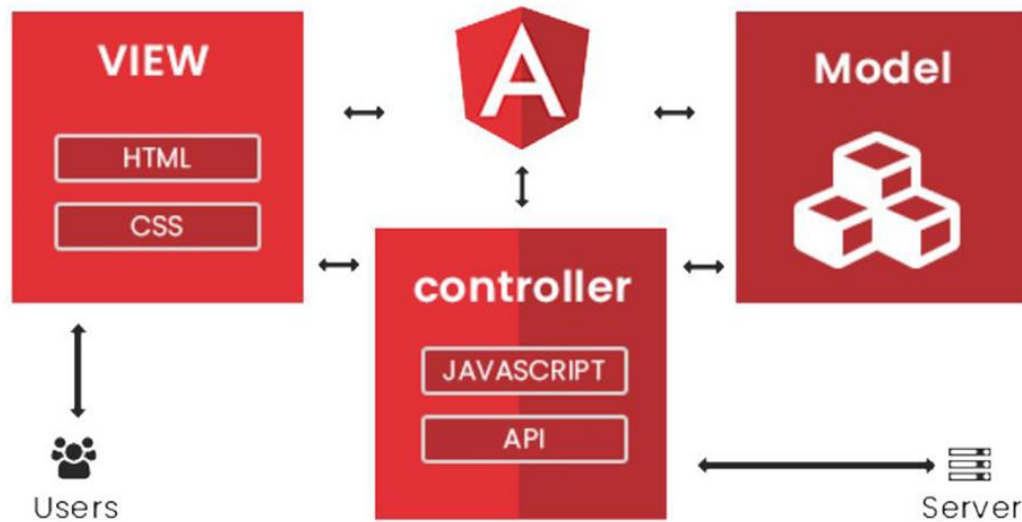
AngularJS developers can quickly build dynamic apps to handle large amounts of data. The abundance of benefits of AngularJS enables content management, handling users and access rights, and applying business rules to become much easier and effortless.

4.1.2 Understanding MVC (Model, View, Controller) architecture

Model View Controller or MVC as it is popularly called, is a software design pattern for developing web applications. A Model View Controller pattern is made up of the following three parts –

- **Model** – It is the lowest level of the pattern responsible for maintaining data.
- **View** – It is responsible for displaying all or a portion of the data to the user.
- **Controller** – It is a software Code that controls the interactions between the Model and View.

MVC is popular because it isolates the application logic from the user interface layer and supports separation of concerns. The controller receives all requests for the application and then works with the model to prepare any data needed by the view. The view then uses the data prepared by the controller to generate a final presentable response. The MVC abstraction can be graphically represented as follows.



The Model

The model is responsible for managing application data. It responds to the request from view and to the instructions from controller to update itself.

The View

A presentation of data in a particular format, triggered by the controller's decision to present the data. They are script-based template systems such as JSP, ASP, PHP and very easy to integrate with AJAX technology.

The Controller

The controller responds to user input and performs interactions on the data model objects. The controller receives input, validates it, and then performs business operations that modify the state of the data model.

AngularJS is a MVC based framework. In the coming chapters, we will see how AngularJS uses MVC methodology.

4.1.2 Setup AngularJS Development Environment

We need the following tools to setup a development environment for AngularJS:

1. AngularJS Library
2. Editor/IDE
3. Browser
4. Web server

1. AngularJS Library

When you open the link <https://angularjs.org/>, you will see there are two options to download AngularJS library –



- **View on GitHub** – By clicking on this button, you are diverted to GitHub and get all the latest scripts.
- **Download AngularJS 1** – By clicking on this button, a screen you get to see a dialog box shown as –
 - Select the required version from the popup and click on download button in the popup.
 - CDN: You can include AngularJS library from CDN url - <https://ajax.googleapis.com/ajax/libs/angularjs/1.3.16/angular.min.js>



2. Editor

AngularJS is eventually HTML and JavaScript code. So you can install any good editor/IDE as per your choice.

The following editors are recommended:

- Sublime Text
- Visual Studio Code
- Visual Studio
- Aptana Studio 3

3. Web server

Use any web server such as IIS, apache etc., locally for development purpose.

4. Browser

You can install any browser of your choice as AngularJS supports cross-browser compatibility. However, it is recommended to use Google Chrome while developing an application.

First AngularJS Application

Let's create a simple AngularJS web application step by step and understand the basic building blocks of AngularJS.

1. First, create an HTML document with <head> and <body> elements, as show below.

Example: HTML Template

```
<html>
  <head>  </head>

  <body>

    </body>
</html>
```

2. Include angular.js file in the head

Example: Include AngularJS Library

```
<html>
  <head>
    <title>First AngularJS Application</title>

    <script src= "angular.js"></script>

  </head>

  <body></body>

</html>
```

3. Point Html AngularJS app

Next, it is required to tell which part of HTML contains the AngularJS app. You can do this by adding the **ng-app** attribute to the root HTML element of the AngularJS app. You can either add it to the html element or the body element as shown below –

Example: Point to AngularJS app

```
<body ng-app>

</body>
```

Here, we will be creating a simple multiplier application which will multiply two numbers and display the result. User will enter two numbers in two separate textboxes and the result will be displayed immediately.

AngularJS Expressions

- AngularJS binds data to HTML using **Expressions**.
- AngularJS expressions can be written inside double braces: **{{ expression }}**.
- AngularJS expressions can also be written inside a directive: **ng-bind="expression"**.
- AngularJS will resolve the expression, and return the result exactly where the expression is written.
- **AngularJS expressions** are much like **JavaScript expressions**: They can contain literals, operators, and variables.

Example

```
<div ng-app>

    {{ 5 + 5 }} <br>

    <p> {{ "Hi" + " " + "This" + " " + "is" + " " + "AWD" + " " + "world" }} </p>

</div>
```

1. Using numbers

```
<p>Addition of two value: {{a * b}} </p>
```

2. Using Strings

```
<p> {{Hi + " " + Welcome + " " + To + " " + AWD+ " " + world}} </p>
```

```
<p>Hello {{employee.firstname + " " + employee.lastname}}!</p>
```

3. Using Object

```
<p>Employee ID: {{ employee.empid}}</p>
```

4. Using Array

```
<p>Marks(Math): {{marks[3]}}</p>
```

Example-1: Demo1.html (apply ng app to body part)

```
<html>
    <head>
        <title>First AngularJS Application</title>

        <script src="angular.js"></script>

    </head>
```

```
<body ng-app >
    <div>
        {{ 5 + 5 }} <br>
        <p> {{ "Hi" + " " + "This" + " " + "is" + " " + "AWD" + " " + "world" }} </p>
    </div>
</body>
</html>
```

Output:

10

Hi This is AWD world

Example-2: Demo1.html (apply ng app to div part)

```
<html>
    <head>
        <title>First AngularJS Application</title>
        <script src="angular.js"></script>
    </head>
    <body>
        <div>
            {{ 5 + 5 }} <br>
            <p> {{ "Hi" + " " + "This" + " " + "is" + " " + "AWD" + " " + "world" }} </p>
        </div>
        <div ng-app>
            {{ 5 + 5 }} <br>
            <p> {{ "Hi" + " " + "This" + " " + "is" + " " + "AWD" + " " + "world" }} </p>
        </div>
    </body>
</html>
```

Output:

```
{{ 5 + 5 }}
```

```
{{ "Hi" + " " + "Welcome" + " " + "To" + " " + "AWD" + " " + "world" }}
```

```
10
```

```
Hi Welcome To AWD world
```

Example: Demo2.html (Multiplication of two No.)

```
<html>

  <head>

    <title>First AngularJS Application</title>

    <script src="angular.js"></script>

  </head>

  <body ng-app >

    <h1>First AngularJS Application</h1>

    Enter Numbers for Multiply:

    <input type="text" ng-model="a" /> X

    <input type="text" ng-model="b" /> = <span>{{a * b}}</span>

  </body>
</html>
```

The above example is looks like HTML code with some strange attributes and braces such as **ng-app**, **ng-model**, and **{{ }}** (**Expression**). These built-in attributes in AngularJS are called directives.

Output:

First AngularJS Application

Enter Numbers to Multiply: x = 60

AngularJS Directives

AngularJS lets you extend HTML with new attributes called **Directives**. AngularJS also lets you define your own directives.

AngularJS has a set of **built-in directives** which offers functionality to you applications.

Directives are markers (attributes) on a DOM element that tell AngularJS to attach a specific behavior to that DOM element or even transform the DOM element and its children.

Most of the directives in AngularJS are starting with prefix **ng**. It stands for Angular.

1. **ng-app** – This directive starts an AngularJS Application.
2. **ng-model** – This directive defines the model that is variable to be used in AngularJS.
3. **ng-bind** – This directive ng-bind is used for one-way data binding.
4. **ng-init** – This directive initializes application data.
5. **ng-repeat** – This directive repeats HTML elements for each item in a collection.

1. ng-app directive

- The ng-app directive starts an AngularJS Application. It defines the root element.
- It automatically initializes or bootstraps the application when the web page containing AngularJS Application is loaded.
- It is also used to load various AngularJS modules in AngularJS Application.
- ng app can be define by 3 ways :
 - first is without modules name

```
<div ng-app>
```

```
</div>
```

- Second is Modules without any name.

```
<body ng-app="">
```

```
</body>
```

- Third is Module with the name.

```
<body ng-app="myangularapp">
```

```
</body>
```

2. ng-model directive

- With the ng-model directives you can bind the value of an input field to a variable created in AngularJS.
- It is commonly used with form elements like input fields (input, Select, text area), checkboxes, and radio buttons to enable two-way data binding.
- Two-way data binding means that changes in the view will automatically update the model, and changes in the model will automatically reflect in the view.
- The ng-model directive tells AngularJS to replace the content of an HTML element with the value of a given variable, or expression.
- In the following example, we define a model named *name*.

```
<div ng-app = "">  
  <p>Enter your Name: <input type = "text" ng-model = "name"></p>  
</div>
```

3. ng-bind directive

- ng-bind is used for one-way data binding. It evaluates an expression and sets the content of an HTML element to the result of that expression.
- Unlike ng-model, it only updates the view with the value from the model and does not update the model when the view changes.

```
<div ng-app = "">  
  <p>Enter your Name: <input type = "text" ng-model = "name"></p>  
  Name is: <p ng-bind="name"></p>  
</div>
```

Example: Creating AngularJS Application to print Your Name (ng- model, ng-bind, ng-app)

```
<html>  
  <head>  
    <title>AngularJS First Application</title>  
    <script src = "angular.js"></script>  
  </head>  
  <body>  
    <h1>Sample Application</h1>  
    <div ng-app = "">
```

```

        <p>Enter your Name: <input type = "text" ng-model = "name"></p>
        <p>Hello <span ng-bind = "name"></span>!</p>
    </div>
</body>
</html>

```

4. ng-init directive

- The ng-init directive initializes an AngularJS Application data like **variables, arrays, object, array of objects** etc. in the scope.
- It allows you to set initial values for variables directly in the view. However, it is generally recommended to initialize variables in the controller instead of using ng-init.
- It defines the initial value for an AngularJS application and assigns values to the variables.
- It is used to assign values to the variables.

```

<html>
    <script src="angular.min.js"></script>
    <body>
        <div ng-app="" ng-init="myText='Angular'">
            <h1>Hi, {{myText}}</h1>
            <h1>{{myText}} is amazing</h1>
        </div>
    </body>
</html>

```

4. ng-repeat directive

- The ng-repeat directive is used to loop over a collection (array or object) and create HTML elements for each item in the collection.
- It allows you to dynamically generate elements based on the data provided.
- In the following example, we iterate over the array of countries.

```

<div ng-init="Subject=['AWD','ASP','PHP','NT']">
    <p>Subject Details:</p>
    <ul>
        <li ng-repeat="Name in Subject">
            {{ Name }}
        </li>
    </ul></div>

```

4.2.1 Some other Directives

1. **ng-class** - ng-class is a directive in Angular that adds and removes CSS classes on an HTML element.

Syntax <element ng-class="expression"></element>

Example

```
<html lang="en">
```

```
<head>                      <title>ng-class</title>                      </head>
```

```
<style>
```

```
    .pink {  
        color:rgb(255, 255, 255);  
        background-color:rgb(255, 49, 186);  
        padding:20px;  
        font-family:"Cambria";  
    }
```

```
    .aqua {  
        background-color:rgb(6, 255, 255);  
        padding:40px;  
        font-family:Verdana;  
    }
```

```
    .lightgreen {  
        color:rgb(255, 255, 255);  
        background-color:rgb(107, 255, 144);  
        padding:15px;  
        font-family:Arial;  
    }
```

```
    .purplle {
```



```
background-color:rgb(162, 0, 255);

padding:10px;

font-family:Verdana;
}
</style>

<body ng-app="">

    <p>Choose a class:</p>

    <select ng-model="home">

        <option value="">Select</option>

        <option value="pink">Pink</option>

        <option value="aqua">Aqua</option>

        <option value="lightgreen">LightGreen</option>

        <option value="purple">Purple</option>

    </select>

    <div ng-class="home">

        <h1>Directives</h1>

        <ol>

            <li>ng-bind </li>

            <li>ng-model</li>

            <li>ng-app</li>

            <li>ng-init</li>

            <li>ng-repeat</li>

            <li>ng-class</li>

        </ol>

    </div>
</body>

</html>
```

2. **ng-animate**- AngularJS provides animated transitions, with help from CSS.

1. ng-hide-

Syntax

```
<element ng-hide="expression"></element>
```

When used as a CSS class:

```
<element class="ng-hide"></element>
```

Example

```
<html>

  <head>

    <title>ng-animate</title>

  </head>

  <script src="angular.min.js"></script>

  <script src="angular-animate.js"></script>

  <style>

    div {

      transition: all linear 0.5s;
      background-color: lightgreen;
      height: 100px;
    }

    .ng-hide {

      height: 0;
    }

  </style>

  <body ng-app="ngAnimate">
    <h1>Check me to Hide: <input type="checkbox" ng-
      model="myCheck"></h1>

    <div ng-hide="myCheck"></div>

  </body>
</html>
```

3. **ng-show**-The ng-show directive shows the specified HTML element if the expression evaluates to true, otherwise the HTML element is hidden.

Syntax

```
<element ng-show="expression"></element>
```

Example

```
<html>

<script src="angular.min.js"></script>
<script src="angular-animate.js"></script>
<title>ng-show</title>
<style>
div {
  transition: all ease-in;
  background-color: lightpink;
  opacity: 50%;
  height: 100px;
}.ng-show {
  height: 0px;
}
</style>
<body ng-app="">
  <h1>Check me To Show <input type="checkbox" ng-model="myCheck" ></h1>
  <div ng-show="myCheck"></div>
</body>
</html>
```

4.2.2 AngularJS - Controllers

AngularJS application mainly relies on controllers to control the flow of data in the application. A controller is defined using *ng-controller* directive. A controller is a JavaScript object that contains attributes/properties, and functions. Each controller accepts *\$scope* as a parameter, which refers to the application/module that the controller needs to handle.

Syntax:

```
<div ng-app = "" ng-controller = "controller_name">  
    //code  
</div>
```

Example:-1

```
<html>  
    <head>  
        <title>AngularJS Controller</title>  
        <script src = " angular.min.js"></script>  
    </head>  
    <body>  
        <h2>Example of AngularJS Controllers </h2>  
        <div ng-app="details" ng-controller="employeecontroller">  
            Id: <input type="text" ng-model="id"><br><br>  
            Name: <input type="text" ng-model="name"><br><br>  
            Details of Employee: {{id + " " + name}}  
        </div>  
        <script>  
            var e = angular.module('details', []);  
            e.controller('employeecontroller', function($scope) {  
                $scope.id = 20;  
                $scope.name = "abc";  
            });  
        </script>
```

```
</body>
</html>
```

Output

Example of AngularJS Controllers

Id:

Name:

Details of Employee: 20 abc

How to define a controller in AngularJS?

There are two ways in which a controller can be defined in an AngularJS application. One is by defining it as an application module and the second way is by defining it as a JavaScript function.

An Application Module

A controller can be defined as an application module.

Syntax:

```
angular.module("myapp", [])
.controller("appController", function($scope) {
    // definition of controller
});
```

Example:

```
<html>
  <head>
    <title> Controllers In AngularJS</title>
    <script src="angular.js"></script>
  </head>
  <body ng-app="myapp">
    <div> Application.controller</div>
```

```
<div ng-controller="appController">
    <div> My Name (app.contrller): {{myName}} </div>
</div><br/>
<script>
    angular.module("myapp", [])
    .controller("appController", function($scope)
    {
        $scope.myName = "PHP";
    });
</script>
</body>
</html>
```

Output:

Application.controller

My name (app.controller): PHP

JavaScript Function

AngularJS controller can also be defined as a JavaScript function.

Syntax:

```
function controllerAsFunction($scope){
    //definition of controller
}
```

Example:

```
<html>
    <head>
        <title> Controllers and Services In AngularJS</title>
        <script src="angular.js"></script>
    </head>
```

```
<body ng-app="myapp">
  <div> Controller as function</div>
    <div ng-controller="Controller As Function">
      <div> My Name (controller as function): {{myName}} </div>
    </div>
  <script>
    function controllerAsFunction($scope) {
      $scope.myName = "php";
    }
  </script>
</body>
</html>
```

Output:

Controller As Function

My Name (controller as function): php

Purpose of AngularJS Controller

The AngularJS Controller has two purposes. It sets the initial value of \$scope object and attaches the behavior to it. The primary responsibility of an AngularJS Controller is to control the data that gets passed to the view. There is two-way communication between the scope and the view .

AngularJS Controller in External Files

The AngularJS Controller can be stored in external files in case of a larger application. An external file needs to be created and saved with the extension, .js.

For example, file_name.js can be the external file that you can create. Its path can be provided with the help of the source attribute in a script tag.

```
<html>
  <script src=" angular.min.js"></script>
```

```
<body>
  <div ng-app="myApp" ng-controller="personCtrl">
    Enter Name: <input type="text" ng-model="fName"><br><br>
    Tutorial Name: {{fullName()}}
  </div>
  <script src="controller.js"></script>
</body>
</html>
```

The code below should be saved as controller.js

```
var app = angular.module('myApp', []);
app.controller('personCtrl', function($scope) {
  $scope.fullName = function() {
    return $scope.fName;
  };
});
```

Output:

Enter Name: php

Tutorial Name: php

AngularJS Scopes

- The Scope is an object that is specified as a binding part between the HTML (view) and the JavaScript (controller). It plays a role of joining controller with the views. It is available for both the view and the controller.

How to use Scope

To make a controller in AngularJS, you have to pass the \$scope object as an argument.

Example-2

```
<html lang="en">
<head>      <title>Controller</title>
            <script src="angular.min.js"></script>
</head>
<body>
<div ng-app="mymodule" ng-controller="mycontrol">
    <!--view data part-->      <!--One way data binding -->
        <h1> Name: {{ Name }}</h1>
        <h2 ng-bind="Name"> </h2>
    <!-- Two Way data binding-->
        Address: <input type="text" ng-model="Address"><br><br><br>
        <h2>{{Address}}</h2><br><br>
    <!-- Two Way data binding Throgh Object-->
    <table>
        <tr>
            <td> Name:</td>
            <td> <input type="text" ng-model="Employee.name"></td>
        </tr>
        <tr>
            <td> Address:</td>
            <td><input type="text" ng-model="Employee.address"> </td>
```

```
</tr>
<tr>
  <td> City: </td>
  <td> <input type="text" ng-model="Employee.city"> </td>
</tr>
</table><br>
<table border="1">
  <tr>
    <th> Name:</th>
    <td> {{Employee.name}}</td>
  </tr>
  <tr>
    <th> Address:</th>
    <td>{{Employee.address}} </td>
  </tr>
  <tr>
    <th> City: </th>
    <td> {{Employee.city}} </td>
  </tr>
</table>
</div>
<!--Model data part-->
<script>
  var app = angular.module('mymodule', []);
  app.controller('mycontrol', function($scope) {
    $scope.Name = "pooja";
    $scope.Address = "Adajan";
```

```
$scope.Employee= {name:"pooja",address:"adajan",city:"surat"};

});

</script>

</body>

</html>
```

4.2.3 AngularJS - Filters

Filters can be added in AngularJS to format data to display on UI without changing the original format. Filters can be added to an expression or directives using the pipe | symbol.

Syntax

```
{{expression | filterName:parameter }}
```

The following list shows the commonly used filters.

Name	Name & Description
uppercase	converts a text to upper case text.
lowercase	converts a text to lower case text.
currency	formats text in a currency format.
filter	filter the array to a subset of it based on provided criteria.
orderby	orders the array based on provided criteria.

1. Uppercase Filter

Add uppercase filter to an expression using pipe character. Here we've added uppercase filter to print student name in all capital letters.

Example:

```
<div ng-app="" ng-init="Name=php">
```

```
<p>Name: {{ Name | uppercase }}</p>
</div>
```

Output:

Name: PHP

2. Lowercase Filter

Add lowercase filter to an expression using pipe character. Here we've added lowercase filter to print student name in all lowercase letters.

Example:

```
<div ng-app="" ng-init="Name=PHP">
  <p>Name: {{ Name | lowercase }}</p>
</div>
```

Output:

Name:php

3.Currency Filter

Add currency filter to an expression returning number using pipe character. Here we've added currency filter to print fees using currency format.

Example:

```
<div ng-app="" >
  Salary: <input type="number" ng-model="salary">
  <p>Total = {{salary | currency }}</p>
```

Output:

Salary: 500
Total = \$500.00

4.Filter

To display only required subjects, we use subjectName as filter.

Example:

```
<html>
```

```
<head>
  <script src = " angular.min.js"></script>
</head>
<body>
  <div ng-app = "details" ng-controller = "studentController">
    Enter subject: <input type = "text" ng-model = "subjectName">
    <ul>
      <li ng-repeat = "subject in student.subjects | filter: subjectName ">
        {{ subject.name + ', marks:' + subject.marks }}
      </li>
    </ul>
  </div>
<script>
  var details= angular.module("details", []);
  details.controller('studentController', function($scope) {
    $scope.student = {
      subjects:[ {name:'ASP',marks:85},
        {name:'PHP',marks:90},
        {name:'AWD',marks:80},
        {name:'NT',marks:75}
      ]};
  });
</script>
</body>
</html>
```

5. OrderBy Filter

To order subjects by marks, we use orderBy marks.

Subject:

```
<ul>
```

```
  <li ng-repeat = "subject in student.subjects | orderBy:'marks'">
```

```
    {{ subject.name + ', marks:' + subject.marks }}
```

```
  </li> </ul>
```

6. Filters Filter

To Filter Fruits by given specific value, we use **Filter** fruite.

```
<html lang="en">
```

```
<head>
```

```
  <script src="angular.min.js"></script>
```

```
  <title>Filters</title>
```

```
  <style>
```

```
    .purplle {
```

```
      color:rgb(81, 2, 126);
```

```
      padding:10px;
```

```
    }.aqua {
```

```
      color:rgb(42, 146, 1);
```

```
      padding:10px;
```

```
  }
```

```
  </style>
```

```
</head>
```

```
<body ng-app=" myfruits ">
```

```
<div ng-init=" mycontrol ">
```

```
  <ul>
```

```

    <li ng-repeat="fruit in Fruits">
        <h3>{{ fruit}}</h3>
    </li>

    <li ng-repeat="fruit in Fruits | filter:'Mango'">
        <h3 class="purplle">{{ fruit }}</h3>
    </li>

    <li ng-repeat="fruit in Fruits | filter:'ap'">
        <h3 class="aqua">{{ fruit }}</h3>
    </li>
</ul>
</div>

<script>

var app = angular.module('myfruits', []);

app.controller('mycontrol', function($scope) {

    $scope.Fruits=
    ["Apple","Apricots","Banana","Blueberries","Cherries","Mango","Strawberries","Grap
s","Pineapple","Muskmelon"]; });

</script>

</body>

</html>

```

7. limitTo Filter

```

<html lang="en">

<head>

    <script src="angular.min.js"></script>

```

```
<title>Filters</title>

</head>

<body ng-app="myfruits">

  <div ng-controller="mycontrol" >

    <ul>

      <li ng-repeat="fruit in Fruits">

        {{fruit}}

      </li>

    </ul>

    Enter Number:<input type="text" placeholder="Enter Limit Number" ng-
model="number"><br> <br>

    <table border="1">

      <tr>

        <th> Name </th>

        <th> Address</th>

        <th> Age </th>

        <th> Salary</th>

      </tr>

      <tr ng-repeat="emp in Employee | limitTo:number">

        <td> {{emp.name}} </td>

        <td> {{emp.address}}</td>

        <td>{{emp.age}}</td>

        <td> {{emp.salary}} </td>

      </tr>
```



```
</table>

</div>

<script>

var app = angular.module('myfruits', []);

app.controller('mycontrol', function($scope) {

    $scope.Fruits=
    ["Apple","Muskmelon","Graps","Blueberries","Cherries","Mango","Apricots","Strawb
erries","Banana","Pineapple"];

    $scope.Employee= [{name:"Asha",address:"Bhatar",age:32,salary:10000},

        {name:"Nisha",address:"Nanpura",age:34,salary:25000},

        {name:"Nidhi",address:"Vesu",age:28,salary:38000},

        {name:"Kiran",address:"Citylight",age:29,salary:18000},

        {name:"Karan",address:"Althan",age:34,salary:25000},

        {name:"Parth",address:"Vesu",age:28,salary:34000},

        {name:"Aman",address:"Adajan",age:29,salary:16800}]; });

</script>

</body></html>
```

AngularJS - Tables

Table data is generally repeatable. The ng-repeat directive can be used to draw table easily. The following example shows the use of ng-repeat directive to draw a table –

```
<html>
<head>
    <title>Angular JS Table</title>
    <script src = "angular.min.js"></script>

    <style>
        table, th , td {
            border: 1px solid grey;
```

```
border-collapse: collapse;
padding: 5px;
}
table tr:nth-child(odd) {
background-color: #f2f2f2;
}
table tr:nth-child(even) {
background-color: #ffffff;
}
</style>
</head>

<body>
<h2>AngularJS Sample Application</h2>
<div ng-app = "mainApp" ng-controller = "studentController">

<table border = "0">
<tr>
<td>Enter first name:</td>
<td><input type = "text" ng-model = "student.firstName"></td>
</tr>
<tr>
<td>Enter last name: </td>
<td>
<input type = "text" ng-model = "student.lastName">
</td>
</tr>
<tr>
<td>Name: </td>
<td>{{student.fullName()}}</td>
</tr>
<tr>
<td>Subject:</td>

<td>
<table>
<tr>
<th>Name</th>
<th>Marks</th>
</tr>
</td>
</tr>
</table>
</div>
</body>
</html>
```

```
        <tr ng-repeat = "subject in student.subjects">
            <td>{{ subject.name }}</td>
            <td>{{ subject.marks }}</td>
        </tr>
    </table>
</td>
</tr>
</table>
</div>

<script>
    var mainApp = angular.module("mainApp", []);

    mainApp.controller('studentController', function($scope) {
        $scope.student = {
            firstName: "",
            lastName: "",
            fees:500,

            subjects:[
                {name:'Physics',marks:70},
                {name:'Chemistry',marks:80},
                {name:'Math',marks:65},
                {name:'English',marks:75},
                {name:'Hindi',marks:67}
            ],
            fullName: function() {
                var studentObject;
                studentObject = $scope.student;
                return studentObject.firstName + " " + studentObject.lastName;
            }
        };
    });
</script>

</body>
</html>
```